Question 1 — Télécharger le dépôt GitHub sur ton ordinateur. (0,5 pt)

```
git clone <URL_DU_DEPOT>
```

Question 2 — Afficher les URL des dépôts distants. (0,5 pt)

```
git remote -v
```

Question 3 — Récupérer et intégrer les modifications du dépôt distant. (0,5 pt)

```
git pull
```

Question 4 — Afficher l'historique des commits sous forme courte. (0,5 pt)

```
git log --oneline
```

Question 5 — Afficher les différences entre les fichiers modifiés et le dernier commit. (1 pt)

```
git diff
```

Question 6 — Ajouter uniquement src/App.jsx à la zone de staging. (0,5 pt)

```
git add src/App.jsx
```

Question 7 — Ajouter toutes les modifications du projet à la zone de staging, y compris les suppressions. (0,5 pt)

```
git add -A
```

Question 8 — Créer un commit avec le message Ajout de la recherche. (0,5 pt)

```
git commit -m "Ajout de la recherche"
```

Question 9 — Envoyer les commits locaux sur GitHub. (0,5 pt)

```
git push
```

Question 10 — Afficher les détails du dernier commit. (1 pt)

```
git show HEAD
```

Question 11 — Supprimer debug.log du disque et préparer sa suppression dans Git. (1 pt)

```
git rm debug.log
```

Question 12 — Annuler les modifications locales de src/App.jsx sans toucher aux autres fichiers. (1 pt)

```
git restore src/App.jsx
```

Question 13 — Retirer src/App.jsx du staging tout en conservant ses modifications. (1 pt)

git restore --staged src/App.jsx

Question 14 — Annuler toutes les modifications non commitées des fichiers suivis et vider le staging. (1 pt)

git reset --hard HEAD

Question 15 — Annuler le dernier commit en conservant les modifications dans le staging. (1 pt)

git reset --soft HEAD~1

Question 16 — Annuler le dernier commit en conservant les modifications dans les fichiers, sans staging. (1 pt)

git reset --mixed HEAD~1

Question 17 — Annuler le dernier commit en conservant les modifications dans le staging, puis vérifier l'historique. (1 pt):

git reset --soft HEAD~1

git log --oneline

Question 18 — Annuler un commit déjà partagé sur GitHub sans supprimer l'historique existant. (0,5 pt)

git revert <hash-du-commit>

git push

Remplace <hash-du-commit> par l'identifiant du commit à annuler.

Question 19 — Afficher les branches locales. (1 pt)

git branch

Question 20 — Créer une branche feature/search sans changer de branche. (1 pt)

git branch feature/search

Question 21 — Basculer vers feature/search avec git switch. (1 pt)

git switch feature/search

Question 22 — Créer et rejoindre feature/profile avec une seule commande git switch. (1 pt)

git switch -c feature/profile

Question 23 — Revenir sur la branche main avec git checkout, puis avec git switch. (1 pt)

git checkout main

git switch main

Question 24 — Supprimer la branche locale feature/header. (1 pt)

git branch -d feature/header

Cette commande supprime une branche déjà fusionnée. Pour forcer la suppression d'une branche non fusionnée, on peut utiliser git branch -D feature/header, mais cela peut faire perdre des commits non fusionnés.

Question 25 — Afficher les URL du dépôt distant. (0,5 pt)

git remote -v

Git reset –hard Vs git revert ID

git reset --hard HEAD~1

- Annule le dernier commit en déplaçant la branche vers le commit précédent.
- Supprime les modifications concernées du staging et des fichiers suivis.
- Peut faire perdre des modifications non sauvegardées.
- À éviter sur une branche partagée, car il réécrit l'historique de la branche.

git revert <id>

- Annule les changements introduits par le commit identifié.
- Crée un nouveau commit d'annulation.
- Conserve l'historique existant.
- C'est généralement la méthode à privilégier pour annuler un commit déjà partagé sur GitHub.

Comparaison rapide

`git reset --hard`	`git revert C`
Avant : A → B → C	Avant : A → B → C
Après : A → B	Après : A → B → C → D
C n'est plus dans l'historique de la branche.	D annule les changements de C.